

The 4 Pillars of OOP

The four pillars of **Object-Oriented Programming (OOP)** are fundamental concepts that help organize and structure code, making it reusable, scalable, and easy to maintain.

1. **Encapsulation** (Hiding Data)
2. **Abstraction** (Hiding Implementation)
3. **Inheritance** (Parent Child)
4. **Polymorphism** (Many Forms)

Encapsulation: The practice of bundling related data and methods into a single unit (a class) and **restrict direct access to some of the object's components, which is a concept known as data hiding.**

Analogy: Think of a medical capsule. The active medicine (data) is hidden inside the outer shell (class). You consume the whole capsule, rather than interacting with the raw medicine directly.

Why it matters: It prevents outside code from accidentally changing or breaking the internal state of an object. You interact with the data only through safely controlled methods (like "**getters**" and "**setters**").

```
class Car {  
  private:  
    int fuel; // Hidden data  
  
  public:  
    // Setter: Controls how data is modified  
    void setFuel(int f) {  
      if (f >= 0) fuel = f; // Validation  
    }  
  
    // Getter: Controls how data is accessed  
    int getFuel() { return fuel; }  
};
```

Under the Hood: If an interviewer asks why we use Encapsulation, they are looking for a specific buzzword: **Class Invariants**. An invariant is a rule that must always be true (e.g., "A fuel tank cannot be negative"). Encapsulation forces all external code to go through your public methods (like **setFuel**), ensuring those rules are never broken.

Abstraction involves hiding complex, underlying implementation details and exposing only the essential, relevant features to the user. It allows a programmer to focus on what an object does rather than how it does it.

Analogy: When you drive a car, you use the steering wheel, accelerator, and brakes. You don't need to know how the fuel injection system works or how the spark plugs ignite the fuel. The complex engine mechanics are abstracted away from the driver.

Why it matters: It reduces complexity and increases efficiency by allowing programmers to use pre-built components without needing to understand their intricate internal workings.

```
class Car {
public:
    // Abstraction: You only need to know how to call "start."
    void start() {
        injectFuel();
        igniteSpark();
        crankEngine();
        cout << "Car is ready to drive!";
    }

private:
    // Implementation: These complex details are hidden from the user
    void injectFuel() { /* logic */ }
    void igniteSpark() { /* logic */ }
    void crankEngine() { /* logic */ }
};

// Usage:
myCar.start(); // The user only sees this simple action.
```